

Reporte de Funcionalidad: Borrado Remoto vía FCM

1. Objetivo

Cumplir íntegramente con los requerimientos de seguridad para implementar un sistema de borrado remoto (remote wipe) de datos sensibles, a través de Firebase Cloud Messaging, orientado a un usuario específico y de forma automatizada (primer y segundo plano).

2. Descripción de la actividad

La actividad consistió en tomar el repositorio de base, inicializar correctamente los servicios de Firebase, agregar dependencias clave (`cloud_firestore` , `firebase_auth` , etc.), reescribir la capa de `SecureStorageService` para alojar 4 valores críticos y finalmente programar la lógica del receptor (`RemoteWipeService`) y el emisor (`remote_wipe_sender`) validando la identidad del usuario y la caducidad del token.

3. Estado inicial del repositorio

- Existía un intento de uso de FCM (dependencias agregadas).
- La lógica de Firestore estaba ausente.
- El almacenamiento usaba otros campos (`user_email` , `phone_number`).
- El servicio Remote Wipe identificaba a los usuarios mediante `userHash` y enviaba mensajes por "topics", en contraposición al requerimiento de asociar a usuario específico y sin topics.

4. Tecnologías utilizadas

- **Flutter:** Frontend Android/iOS.
- **Firebase Cloud Messaging (FCM):** Infraestructura push.
- **Firebase Firestore:** Base de datos de registros (tokens-dispositivos).
- **Firebase Auth:** Autenticación anónima para asegurar la DB.
- **Flutter Secure Storage:** Almacenamiento local cifrado.
- **Node.js + Firebase Admin SDK:** Herramienta de emisión de consola externa.

5. Arquitectura de la solución

```
Node.js Script (Admin SDK) --> FCM Backend --> Flutter App (Dispositivo)
      |                                     |
      +--> (Consulta Firestore por Tokens) <-----+
                                           (Registra Token al iniciar
sesión)
```

- **LoginViewModel:** Llama a guardar los 4 datos y solicita el alta en Firestore al loguearse.
- **DeviceRegistrationService:** Recoge el `fcmToken` , verifica el `installationId` guardado y lo sube bajo `users/admin/devices/...`
- **RemoteWipeMessageParser:** Evalúa integridad del payload JSON y fecha de expiración.
- **RemoteWipeService:** Efectúa comparaciones (userId actual vs objetivo, duplicidad de commandId).
- **SecureStorageService:** Controla la escritura cifrada (Android EncryptedSharedPreferences) y el borrado de los 4 campos.

6. Configuración de Firebase

Se habilitó Firestore y Firebase Auth Anónimo en el proyecto destino para que los dispositivos sin sesión en backend puedan persistir su token con seguridad. El APK de validación usa la configuración en `firebase_options.dart`.

7. Implementación del almacenamiento seguro

Se migró de la vieja clase `SecureSensitiveStorage` a `SecureStorageService`. Proporciona `deleteSensitiveData()` el cual remueve exclusivamente las 4 llaves sensibles listadas.

8. Descripción de los cuatro campos sensibles

1. **user_id**: El identificador `admin`.
2. **access_token**: Generado aleatoriamente (32 bytes base64) y simulando JWT.
3. **refresh_token**: Cadena aleatoria de renovación simulada.
4. **session_secret**: Llave efímera de sesión criptográfica.

9. Registro del usuario y token FCM

Los tokens se graban con `DeviceRegistrationService` en:

`users/{userId}/devices/{installationId}` incluyendo la plataforma, para segmentar correctamente en qué celular eliminar la data.

10. Flujo de envío individual

El script `send_remote_wipe.mjs` lee dichos tokens (ignorando inactivos), arma un JSON de tipo `data` y lo despacha con prioridad "High".

11. Validaciones de seguridad

- `action == 'remote_wipe'`.
- `targetUserId == localUserId`.
- `command.isExpired == false` (Máximo 5 minutos de validez desde la emisión).
- Que el `commandId` no sea igual al último procesado (Replay Attack prevention).

12. Procesamiento en primer y segundo plano

La directiva `@pragma('vm:entry-point')` en `main.dart` asegura que si la app está en segundo plano (background), Flutter lanza un *isolate* que puede instanciar el almacenamiento y borrar los datos antes de que el usuario vuelva a abrir la app.

13. Borrado remoto

Se ejecutan comandos de `delete` directos sobre `FlutterSecureStorage` para las 4 variables. Las preferencias normales (inactividad, sesión global) no se rompen.

14. Pruebas realizadas

Se implementaron en `test/features/...` validando:

1. Parseo estricto del mensaje.
2. Invalidez de un mensaje sin CommandId.
3. Mensajes Expirados detectados.
4. Escritura y eliminación exitosa en el mock Storage.

15. Procedimiento de demostración

[Detallado en `docs/GUIA_DEMOSTRACION.md`]

16. Resultados

- `flutter test` : OK (Todos pasaron).
- `flutter analyze` : 0 Errores.
- `flutter build apk` : Completado con éxito.

17. Limitaciones

- En Android 12+, el "Force Stop" evita la entrega de mensajes `data` background. No es error de código.
- Firebase Auth puede requerir conexión previa de red para la creación anónima.

18. Conclusiones

Se logró desvincular el proyecto base de tópicos globales inseguros, acercando el modelo de seguridad a un "Mobile Device Management" (MDM) de nivel Enterprise, donde un servicio Backend dictamina con llaves expirar credenciales comprometidas.

CAPTURA PENDIENTE: Firebase Console > Project settings > General > Android app (Solicitada por la rúbrica si corresponde al dueño). **CAPTURA PENDIENTE:** Firestore Database > Colección `users` -> `admin` -> `devices` evidenciando el guardado.